

saved on to a stack. After a return from the called function, the program counter gets restored for leftover instructions. This is context switching at function level. This consumes time and memory. Needless to say, the function `opt_foo()` is more efficient and faster than `foo()`.

Let's consider another similar example:

```
int foo(int x)
{
    return calc(x) + calc(x);
}

int opt_foo(int x)
{
    return 2 * calc(x);
}
```

You may be tempted to believe that `opt_foo()` is better, because of its less number of calls to `calc()`. Right? Well, before you conclude anything, first have a closer look at function `calc()` to understand what it actually does.

```
int ctr = 0; /* global variable */
int calc (int x)
{
    ctr ++;
    return ctr * x;
}
```

The variable `ctr` in the above snippet is a global variable, and `calc()` modifies it. Hence, `foo()` and `opt_foo()` do not compute the same result. For example, let's assume that we call `foo()` with 5 as the argument. `foo()` will return 15 after calculation. However, `opt_foo()` with 5 as its argument will return 10.

So, the moral here is, although unnecessary function calls should be avoided, a careful logical analysis is also necessary. Otherwise, you may land in trouble with unstable and unpredictable behaviour of the system.

Use of unsigned integers

If you are sure that the value will never go negative, using unsigned integers is advisable, as processors can handle an unsigned integer better than a signed one.

Most of the mainstream C/C++

compilers, by default, implicitly consider an integer declaration as signed—i.e., `int x` will be considered as `signed int x`. Therefore, it's a good practice to always declare such variables explicitly to avoid any confusion.

Branch prediction

Another technique to tune your code and get a performance boost is branch prediction. It is the software's ability to hint to the processor which way a branch (the *if-then-else* structure) is about to go.

According to Wikipedia, “A branch predictor tries to guess which way a branch will go before this is known, for sure. The purpose of the branch predictor is to improve the flow in the instruction pipeline. Branch predictors are crucial in today's pipelined microprocessors for achieving high performance.”

To programmers, the GCC also provides a built-in function `__builtin_expect()` and based on its two macros—`likely()` and `unlikely()`—indicates the branch prediction information to the compiler. This way the compiler generates a more intelligent binary where the expected result is favoured.

The following lines of code demonstrate the use of the `likely()` and `unlikely()` macros:

```
if (unlikely(x != NULL))
    foo();
if (likely(y != NULL))
    opt_foo();
```

Memory alignment and padding

Quoting Wikipedia again, “A memory access is said to be aligned when the datum being accessed is *n* bytes long and the datum address is *n*-byte aligned. When a memory access is not aligned, it is said to be misaligned.”

An aligned memory access will have a less number of memory access cycles than misaligned memory access. Therefore, to ensure that memory is correctly aligned some irrelevant bytes are inserted by the compiler. This activity is known as padding.

With data types like `struct`, any standard C/C++ compiler tries to add padding. Let's consider the following:

```
typedef struct{
    char c;
    int i;
    short s;
}example_1;

typedef struct{
    char c;
    short s;
    int i;
}example_2;
```

Considering that a `char` takes 1 byte, a `short` takes 2 bytes, and an `int` takes 4 bytes, `example_1` will consume 12 bytes. However, `example_2` will only take 8 bytes. This is because, in the above code snippet, the compiler tries to align the structure in the power of the size of an integer.

Hence, in the first case (`example_1`), three 4 bytes are assigned. However, in the second case (`example_2`), since a `char` and a `short` is accommodated within a single 4-byte boundary, we have a structure size of 8 bytes (4 bytes for `char` and `short`, and 4 bytes for `int`). Using these kinds of small tweaks you can always save significant amount of memory.

For further details on this topic, visit en.wikipedia.org/wiki/Data_structure_alignment.

A final note

I've only discussed a few common techniques here. There are many other ways (including many processor-dependent techniques) to fine-tune your code. I hope the examples in this article will inspire you to explore them on your own. 

Resources

- Wikipedia on branch predictor: en.wikipedia.org/wiki/Branch_predictor
- Wikipedia on data structure alignment: en.wikipedia.org/wiki/Data_structure_alignment

By: Vivek Kumar Prasad

The author is currently working as a technical lead at Symphony Services and is based in Pune. He's a Linux enthusiast and you can reach him at mail_DOT_vivekkumar_AT_rediffmail_DOT_com.